

MAT4110 OBLIG 1

Zejing Wang

September 2024

1 How to solve the problem

I will use NumPy in Python to solve the problem and use Matplotlib to plot the figure. QR Decomposition. I will use `numpy.linalg.qr` to perform the QR decomposition, decomposes the matrix A to an orthogonal matrix Q and an upper triangular matrix R . Use back substitution to solve the linear system $Rx = Q^T b$, and get the polynomial coefficients. Finally, use Matplotlib to plot the original noise data and the fitted polynomial.

Cholesky Factorization, create the dataset using a given function with added noise. Use `np.vander` in Python to construct the matrix A for polynomial fitting. Since A is not necessarily symmetric or square, we need compute $A^T A$, which is symmetric and positive definite. Decompose $A^T A$ to LDL^T using Cholesky decomposition. Solve the system using forward and backward substitution. Finally, use Matplotlib to visualize the original data points and the fitted polynomial.

2 Problem 1

This is the code of my python script . and figure is the plot.

2.1 Problem 1 data1

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 # Initial data
4 n = 30
5 start = -2
6 stop = 2
7 x_vals = np.linspace(start, stop, n) # Generate interval
8 eps = 1
9 np.random.seed(1) # Random seed
10 r = np.random.rand(n) * eps # Noise
11 # Function
12 y_vals = x_vals * (np.cos(r + 0.5 * x_vals**3) + np.sin(0.5 * x_vals
13                 **3))
14 # Order
15 m = 3
16 # Vandermonde matrix A
17 A = np.vander(x_vals, m, increasing=True)
18 # QR decomposition
```

```

19 Q, R = np.linalg.qr(A)
20
21 b = y_vals.reshape(-1, 1)
22 Qb = np.dot(Q.T, b)
23
24 # Solve for polynomial coefficients using back substitution
25 x_coeffs = np.zeros((m, 1))
26 for i in range(m - 1, -1, -1):
27     x_coeffs[i] = (Qb[i] - np.dot(R[i, i+1:], x_coeffs[i+1:])) / R[i, i
28 ]
29
30 print('Polynomial coefficients:')
31 print(x_coeffs.flatten())
32
33 # Plot
34 x_plot = np.linspace(start, stop, 100)
35 y_fit = np.zeros_like(x_plot)
36 for i in range(m):
37     y_fit += x_coeffs[i] * x_plot**i
38
39 plt.scatter(x_vals, y_vals, label="Original data", color="blue")
40 plt.plot(x_plot, y_fit, label=f"Degree {m} polynomial fit", color="red"
41 )
42 plt.legend()
43 plt.title(f"Polynomial fit using QR decomposition (m={m})")
44 plt.show()

```

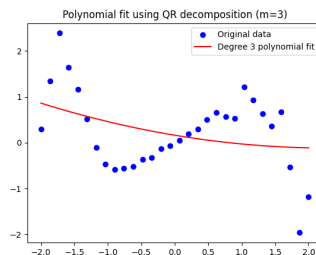


Figure 1: OPPGAVE 1A, m=3, data1.

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Initial data
6 n = 30
7 start = -2
8 stop = 2
9 x_vals = np.linspace(start, stop, n) # Generate interval
10 eps = 1
11 np.random.seed(1) # Random seed
12 r = np.random.rand(n) * eps # Noise
13
14 # Function
15 y_vals = x_vals * (np.cos(r + 0.5 * x_vals**3) + np.sin(0.5 * x_vals
16 **3))
17
18 # Order

```

```

18 m = 8
19
20 # Vandermonde matrix A
21 A = np.vander(x_vals, m, increasing=True)
22
23 # QR decomposition
24 Q, R = np.linalg.qr(A)
25
26 b = y_vals.reshape(-1, 1)
27 Qb = np.dot(Q.T, b)
28
29 # Solve for polynomial coefficients using back substitution
30 x_coeffs = np.zeros((m, 1))
31 for i in range(m - 1, -1, -1):
32     x_coeffs[i] = (Qb[i] - np.dot(R[i, i+1:], x_coeffs[i+1:])) / R[i, i
33 ]
34
35 print('Polynomial coefficients:')
36 print(x_coeffs.flatten())
37
38 # Plot
39 x_plot = np.linspace(start, stop, 100)
40 y_fit = np.zeros_like(x_plot)
41 for i in range(m):
42     y_fit += x_coeffs[i] * x_plot**i
43
44 plt.scatter(x_vals, y_vals, label="Original data", color="blue")
45 plt.plot(x_plot, y_fit, label=f"Degree {m} polynomial fit", color="red")
46 plt.legend()
47 plt.title(f"Polynomial fit using QR decomposition (m={m})")
48 plt.show()

```

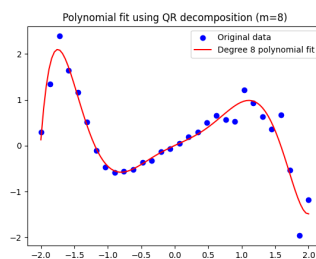


Figure 2: OPPGAVE 1A m=8, cos sin.

2.2 Problem1 Data2

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Initial data
5 n = 30
6 start = -2
7 stop = 2
8 x_vals = np.linspace(start, stop, n) # Generate interval
9 eps = 1

```

```

10 np.random.seed(1) # Random seed
11 r = np.random.rand(n) * eps # Noise
12
13 # Function with noise
14 y_vals = 4 * x_vals**5 - 5 * x_vals**4 - 20 * x_vals**3 + 10 * x_vals
    **2 + 40 * x_vals + 10 + r
15
16 # Order
17 m = 3
18
19 # Vandermonde matrix A
20 A = np.vander(x_vals, m, increasing=True)
21
22 # QR decomposition
23 Q, R = np.linalg.qr(A)
24
25 # Solve for the coefficients using back substitution
26 b = y_vals.reshape(-1, 1)
27 Qb = np.dot(Q.T, b)
28
29 x_coeffs = np.zeros((m, 1))
30 for i in range(m - 1, -1, -1):
31     x_coeffs[i] = (Qb[i] - np.dot(R[i, i+1:], x_coeffs[i+1:])) / R[i, i
    ]
32
33 # Display the polynomial coefficients
34 print('Polynomial coefficients from QR decomposition:')
35 print(x_coeffs.flatten())
36
37 # Plot original data and the fitted polynomial curve
38 x_plot = np.linspace(start, stop, 100)
39 y_fit = np.zeros_like(x_plot)
40 for i in range(m):
41     y_fit += x_coeffs[i] * x_plot**i
42
43 plt.scatter(x_vals, y_vals, label="Original data", color="blue")
44 plt.plot(x_plot, y_fit, label=f"Degree {m} polynomial fit", color="red"
    )
45 plt.legend()
46 plt.title(f"Polynomial fit using QR decomposition (m={m}) - Second
    Dataset")
47 plt.show()

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Initial data
5 n = 30
6 start = -2
7 stop = 2
8 x_vals = np.linspace(start, stop, n) # Generate equally spaced points
9 eps = 1
10 np.random.seed(1) # Radom seed
11 r = np.random.rand(n) * eps # Noise
12
13 # Function with noise
14 y_vals = 4 * x_vals**5 - 5 * x_vals**4 - 20 * x_vals**3 + 10 * x_vals
    **2 + 40 * x_vals + 10 + r

```

```

15
16 # Order
17 m = 8
18
19 # Vandermonde matrix A
20 A = np.vander(x_vals, m, increasing=True)
21
22 # QR decomposition
23 Q, R = np.linalg.qr(A)
24
25 # Back substitution
26 b = y_vals.reshape(-1, 1)
27 Qb = np.dot(Q.T, b)
28
29 x_coeffs = np.zeros((m, 1))
30 for i in range(m - 1, -1, -1):
31     x_coeffs[i] = (Qb[i] - np.dot(R[i, i+1:], x_coeffs[i+1:])) / R[i, i
32 ]
33
34 # Display the polynomial coefficients
35 print('Polynomial coefficients from QR decomposition:')
36 print(x_coeffs.flatten())
37
38 # Plot original data and the fitted polynomial curve
39 x_plot = np.linspace(start, stop, 100)
40 y_fit = np.zeros_like(x_plot)
41 for i in range(m):
42     y_fit += x_coeffs[i] * x_plot**i
43
44 plt.scatter(x_vals, y_vals, label="Original data", color="blue")
45 plt.plot(x_plot, y_fit, label=f"Degree {m} polynomial fit", color="red")
46 plt.legend()
47 plt.title(f"Polynomial fit using QR decomposition (m={m}) - Second Dataset")
48 plt.show()

```

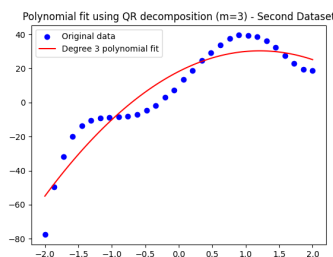


Figure 3: OPPGAVE 1A Data2, $m=3$.

3 Problem 2

Cholesky factorization

3.1 Problem2 Data1

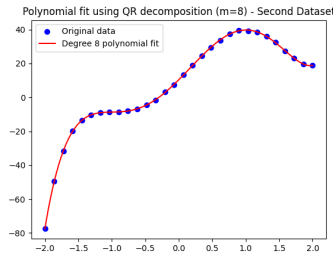


Figure 4: OPPGAVE 1 data 2, $m=8$.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Forward Substitution
5  def forward_substitution(L, b):
6      """ Solve  $L * y = b$  using forward substitution where  $L$  is a lower
7          triangular matrix. """
8      y = np.zeros_like(b)
9      for i in range(len(b)):
10         y[i] = (b[i] - np.dot(L[i, :i], y[:i])) / L[i, i]
11     return y
12
13 # Back Substitution
14 def back_substitution(U, y):
15     """ Solve  $U * x = y$  using back substitution where  $U$  is an upper
16         triangular matrix. """
17     x = np.zeros_like(y)
18     for i in range(len(y)-1, -1, -1):
19         x[i] = (y[i] - np.dot(U[i, i+1:], x[i+1:])) / U[i, i]
20     return x
21
22 # Cholesky Least Squares Implementation
23 def cholesky_least_squares(A, b):
24     # Compute  $A.T @ A$ 
25     ATA = A.T @ A
26     # Compute  $A.T @ b$ 
27     ATb = A.T @ b
28     # Cholesky decomposition:  $ATA = R.T @ R$ 
29     R = np.linalg.cholesky(ATA)
30     # Solve  $R @ y = A.T @ b$  using forward substitution (because  $R$  is
31         lower triangular)
32     y = forward_substitution(R, ATb)
33     # Solve  $R.T @ x = y$  using back substitution (because  $R.T$  is upper
34         triangular)
35     x = back_substitution(R.T, y)
36     return x
37
38 # First dataset
39 n = 30
40 x_data = np.linspace(-2, 2, n)
41 eps = 1
42 np.random.seed(1)
43 r = np.random.rand(n) * eps
44
45 # First dataset

```

```

42 y_data1 = x_data * (np.cos(r + 0.5 * x_data**3) + np.sin(0.5 * x_data
    **3))
43
44 # Construct the Vandermonde matrix A (m = 3)
45 m = 3
46 A = np.vander(x_data, m, increasing=True)
47
48 # Solve least squares for the first dataset
49 coefficients1 = cholesky_least_squares(A, y_data1)
50
51 # Compute the fitted polynomial values
52 fit1 = A @ coefficients1
53
54 # Plot first dataset with m=3
55 plt.figure()
56 plt.scatter(x_data, y_data1, label='Data 1', color='blue')
57 plt.plot(x_data, fit1, label='Fit 1 (m=3)', color='red')
58 plt.title('Polynomial Fit for Data 1 (m=3)')
59 plt.legend()
60
61 plt.show()

1     import numpy as np
2     import matplotlib.pyplot as plt
3
4     # Forward Substitution
5     def forward_substitution(L, b):
6         """ Solve  $L * y = b$  using forward substitution where L is a lower
            triangular matrix. """
7         y = np.zeros_like(b)
8         for i in range(len(b)):
9             y[i] = (b[i] - np.dot(L[i, :i], y[:i])) / L[i, i]
10        return y
11
12    # Back Substitution
13    def back_substitution(U, y):
14        """ Solve  $U * x = y$  using back substitution where U is an upper
            triangular matrix. """
15        x = np.zeros_like(y)
16        for i in range(len(y)-1, -1, -1):
17            x[i] = (y[i] - np.dot(U[i, i+1:], x[i+1:])) / U[i, i]
18        return x
19
20    # Cholesky Least Squares Implementation
21    def cholesky_least_squares(A, b):
22        # Compute  $A.T @ A$ 
23        ATA = A.T @ A
24        # Compute  $A.T @ b$ 
25        ATb = A.T @ b
26        # Cholesky decomposition:  $ATA = R.T @ R$ 
27        R = np.linalg.cholesky(ATA)
28        # Solve  $R.T @ y = A.T @ b$  using forward substitution
29        y = forward_substitution(R.T, ATb)
30        # Solve  $R @ x = y$  using back substitution
31        x = back_substitution(R, y)
32        return x
33
34    # First dataset

```

```

35 n = 30
36 x_data = np.linspace(-2, 2, n)
37 eps = 1
38 np.random.seed(1)
39 r = np.random.rand(n) * eps
40
41 # First dataset
42 y_data1 = x_data * (np.cos(r + 0.5 * x_data**3) + np.sin(0.5 * x_data
    **3))
43
44 # Construct the Vandermonde matrix A ( m = 8)
45 m = 8
46 A = np.vander(x_data, m, increasing=True)
47
48 # Solve least squares for the first dataset
49 coefficients1 = cholesky_least_squares(A, y_data1)
50
51 # Compute the fitted polynomial values
52 fit1 = A @ coefficients1
53
54 # Plot first dataset with m=8
55 plt.figure()
56 plt.scatter(x_data, y_data1, label='Data 1', color='blue')
57 plt.plot(x_data, fit1, label='Fit 1 (m=8)', color='red')
58 plt.title('Polynomial Fit for Data 1 (m=8)')
59 plt.legend()
60
61 plt.show()

```

3.2 Problem2b Data2

```

1     import numpy as np
2     import matplotlib.pyplot as plt
3
4     # Forward Substitution
5     def forward_substitution(L, b):
6         """ Solve L * y = b using forward substitution where L is a lower
            triangular matrix. """
7         y = np.zeros_like(b)
8         for i in range(len(b)):
9             y[i] = (b[i] - np.dot(L[i, :i], y[:i])) / L[i, i]
10        return y
11
12    # Back Substitution
13    def back_substitution(U, y):
14        """ Solve U * x = y using back substitution where U is an upper
            triangular matrix. """
15        x = np.zeros_like(y)
16        for i in range(len(y)-1, -1, -1):
17            x[i] = (y[i] - np.dot(U[i, i+1:], x[i+1:])) / U[i, i]
18            print(U[i, i], y[i], np.dot(U[i, i+1:], x[i+1:]))
19        return x
20
21    # Cholesky Least Squares Implementation
22    def cholesky_least_squares(A, b):
23        # Compute A.T @ A
24        ATA = A.T @ A

```



```

25     # Compute A.T @ b
26     ATb = A.T @ b
27     # Cholesky decomposition: ATA = R.T @ R
28     R = np.linalg.cholesky(ATA)
29     # Solve R @ y = A.T @ b using forward substitution (because R is
        lower triangular)
30     y = forward_substitution(R, ATb)
31     # Solve R.T @ x = y using back substitution (because R.T is upper
        triangular)
32     x = back_substitution(R.T, y)
33     return x
34
35 # Second dataset
36 n = 30
37 x_data = np.linspace(-2, 2, n)
38 eps = 1
39 np.random.seed(1)
40 r = np.random.rand(n) * eps
41
42
43 y_data1 = 4*x_data**5 - 5*x_data**4 - 20*x_data**3 + 10*x_data**2 + 40*
    x_data + 10 + r
44
45 # Construct the Vandermonde matrix A ( m = 3)
46 m = 3
47 A = np.vander(x_data, m, increasing=True)
48
49 # Solve least squares for the first dataset
50 coefficients1 = cholesky_least_squares(A, y_data1)
51
52 # Compute the fitted polynomial
53 fit1 = A @ coefficients1
54
55 # Plot first dataset with m=8
56 plt.figure()
57 plt.scatter(x_data, y_data1, label='Data 2', color='blue')
58 plt.plot(x_data, fit1, label='Fit 1 (m=3)', color='red')
59 plt.title('Polynomial Fit for Data 2 (m=3)')
60 plt.legend()
61
62 plt.show()

```

```

1     import numpy as np
2     import matplotlib.pyplot as plt
3
4     # Forward Substitution
5     def forward_substitution(L, b):
6         """ Solve L * y = b using forward substitution where L is a lower
            triangular matrix. """
7         y = np.zeros_like(b)
8         for i in range(len(b)):
9             y[i] = (b[i] - np.dot(L[i, :i], y[:i])) / L[i, i]
10        return y
11
12    # Back Substitution
13    def back_substitution(U, y):
14        """ Solve U * x = y using back substitution where U is an upper
            triangular matrix. """

```

```

15     x = np.zeros_like(y)
16     for i in range(len(y)-1, -1, -1):
17         x[i] = (y[i] - np.dot(U[i, i+1:], x[i+1:])) / U[i, i]
18         print(U[i, i], y[i], np.dot(U[i, i+1:], x[i+1:]))
19     return x
20
21 # Cholesky Least Squares Implementation
22 def cholesky_least_squares(A, b):
23     # Compute A.T @ A
24     ATA = A.T @ A
25     # Compute A.T @ b
26     ATb = A.T @ b
27     # Cholesky decomposition: ATA = R.T @ R
28     R = np.linalg.cholesky(ATA)
29     # Solve R @ y = A.T @ b using forward substitution (because R is
        lower triangular)
30     y = forward_substitution(R, ATb)
31     # Solve R.T @ x = y using back substitution (because R.T is upper
        triangular)
32     x = back_substitution(R.T, y)
33     return x
34
35 # Second dataset
36 n = 30
37 x_data = np.linspace(-2, 2, n)
38 eps = 1
39 np.random.seed(1)
40 r = np.random.rand(n) * eps
41
42
43 y_data1 = 4*x_data**5 - 5*x_data**4 - 20*x_data**3 + 10*x_data**2 + 40*
    x_data + 10 + r
44
45 # Construct the Vandermonde matrix A ( m = 8)
46 m = 8
47 A = np.vander(x_data, m, increasing=True)
48
49 # Solve least squares for the first dataset
50 coefficients1 = cholesky_least_squares(A, y_data1)
51
52 # Compute the fitted polynomial
53 fit1 = A @ coefficients1
54
55 # Plot first dataset with m=8
56 plt.figure()
57 plt.scatter(x_data, y_data1, label='Data 2', color='blue')
58 plt.plot(x_data, fit1, label='Fit 1 (m=8)', color='red')
59 plt.title('Polynomial Fit for Data 2 (m=8)')
60 plt.legend()
61
62 plt.show()

```

4 Problem 3

When we solving the least squares problems, we can use QR and Cholesky decomposition. These are two useful tools. QR decomposition decomposes the matrix A into

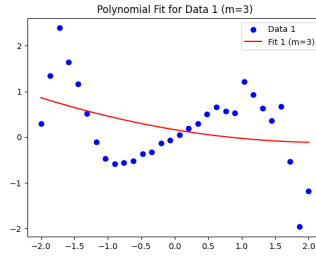


Figure 5: OPPGAVE 2 m=3, data1.

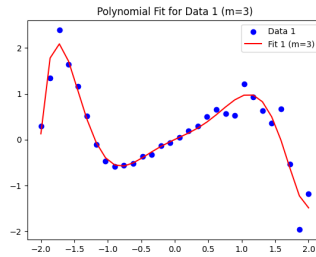


Figure 6: OPPGAVE 2 data1 m=8.

the product of an orthogonal matrix Q and an upper triangular matrix R and Cholesky decomposition decomposes the matrix A into the product of LDL^T , which L is a lower triangular matrix, and D is diagonal matrix.

But QR and Cholesky decomposition both have advantages and disadvantages

QR decomposition has the advantage of greater numerical stability, especially when dealing with larger matrices, making it more stable. Cholesky decomposition is more useful than QR decomposition, but it requires the matrix $A^T A$ to be positive definite, which is typically the case in least squares problems.

QR decomposition is more stable, particularly when we dealing with matrices that have a large condition number. We don't need to compute directly $A^T A$ which can make more errors

Cholesky decomposition operates on the normal equations $A^T A x = A^T b$ which needs squaring the condition number of the matrix. This means that small error can become much bigger, and because of that, we might lose a lot of significant. This will be big problem when the matrix A has a large condition number. Cholesky decomposition will lose the significant when we solving the normal equations.

QR decomposition is more robust because it works directly on the matrix A . is not on $A^T A$. So this means that QR decomposition is highly robust and can handle ill-conditioned matrix than Cholesky decomposition. If we want to using the Cholesky decomposition, we must assume that $A^T A$ is symmetric and positive definite. If the matrix A is close to singular or not positive definite, Cholesky decomposition may fail or make more numerical error. Thus, it is less robust when the matrix is ill-conditioned or the system is near singularity. Cholesky decomposition is less robust and may not work well if the matrix A is not positive definite or near to singularity.

QR Decomposition have more computational cost, but more stable.

Cholesky Decomposition is faster and easier to compute, but less stable.

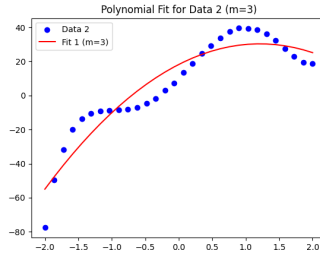


Figure 7: OPPGAVE 2 $m=3$, data2.

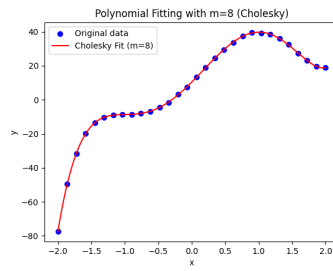


Figure 8: OPPGAVE 2 $m=8$, data2